

File System

Dr. Jit Mukherjee



File

- Attributes -> Name - Some systems give special significance to names, and particularly extensions (.exe, .txt, etc.). Some extensions -> of significance to the OS (.exe), others -> to certain applications (.jpg)
 - **Identifier** (e.g. inode number), **Type** - Text, executable, other binary, etc. **Location** - on the hard drive. **Size**, **Protection**, **Time & Date**, **User ID**
- The file ADT supports many common operations: creating, reading, writing, repositioning within a file, deleting, truncating
- Files be *opened* before access and *closed* after all access.
 - Normally the programmer must open and close files explicitly, but some rare systems open the file automatically at first access.
- Information about currently open files is stored in an *open file table*
 - **File pointer** - records the current position in the file, for the next read or write access
 - **File-open count** - How many times has the current file been opened by different processes and not yet closed? When this counter reaches zero the file can be removed from the table.
 - **Disk location of the file, Access rights**
- **File Locking** - **Shared Lock** (Reading Only), **Exclusive Lock** (Reading and Writing), **Advisory Lock** (Informational not enforced), **Mandatory Lock** (enforced)
 - UNIX used advisory locks, and Windows uses mandatory locks.
 - Advisory lock -> cooperating process

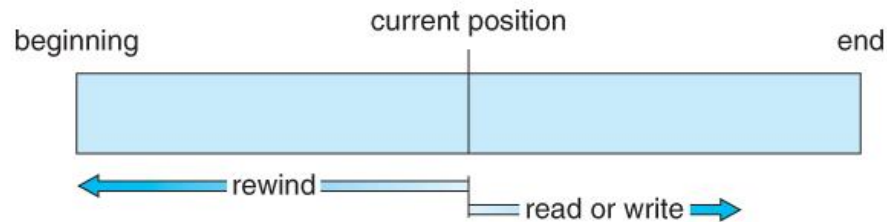
File Contd.

- File type -
 - Macintosh stores a creator attribute for each file, create() system call -> UNIX stores magic numbers at the beginning of certain files.
- File structure - some file contain an internal structure, not known to OS
 - To support a particular file format increase the complexity of OS
 - UNIX -> all file sequence of bytes
 - Mac -> *resource fork*, (UI, button) and a *data fork* (code or data).
- Internal File Structure - Disk files are accessed in units of physical blocks, (512 bytes)
 - Internally organized in units of logical units (1byte to data)
 - The number of logical units -> fit into a physical block -> *packing* -> the amount of internal fragmentation (wasted space).
 - half a physical block is wasted for each file, and the larger the block sizes the more space is lost to internal fragmentation.

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xml, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

Access Method: Sequential and Direct Access

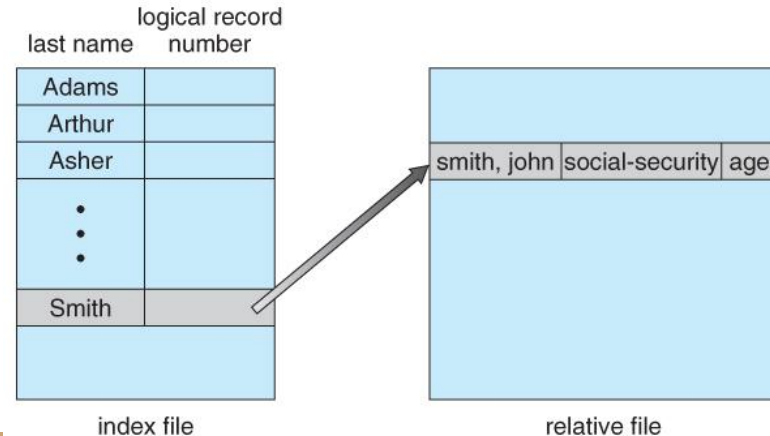
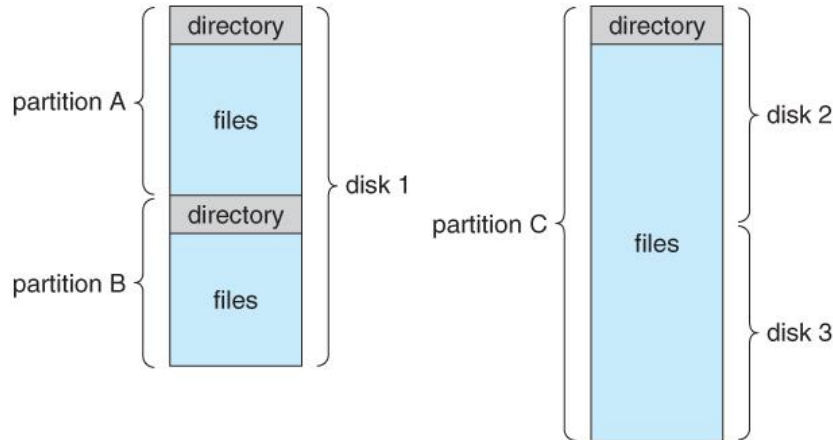
- Sequential Access - file emulates magnetic tape operation
 - read next - read a record and advance the tape to the next position.
 - write next - write a record and advance the tape to the next position.
 - rewind
 - skip N records - May or may not be supported. N may be limited to positive numbers, or may be limited to +/- 1.
- Direct Access - Jump to any record and read that record. Operations supported include:
 - read n - read record number n.
 - write n - write record number n
 - jump to record n - could be 0 or the end of file
 - Query current record - used to return back to this record later.
 - Sequential access can be easily emulated using direct access. The inverse is complicated and inefficient.



sequential access	implementation for direct access
reset	<code>cp = 0;</code>
read_next	<code>read cp ;</code> <code>cp = cp + 1;</code>
write_next	<code>write cp;</code> <code>cp = cp + 1;</code>

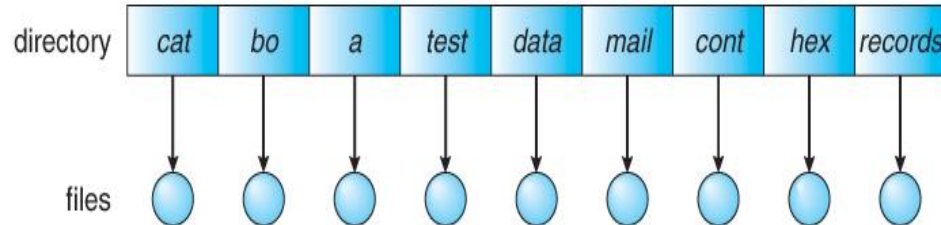
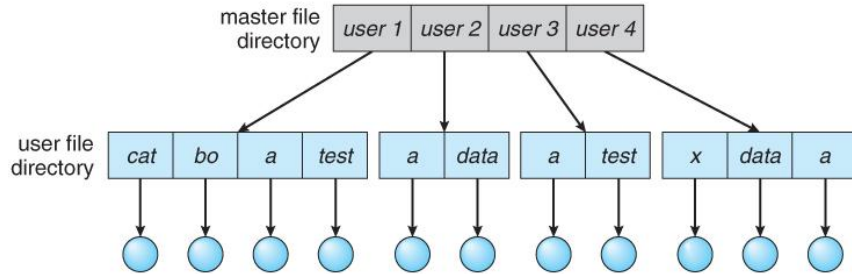
Indexed Access and Directory Structure

- An indexed access scheme can be easily built on top of a direct access system. Very large files may require a multi-tiered indexing scheme, i.e. indexes of indexes.
- Storage Structure - A disk can be used in its entirety for a file system.
 - Alternatively a physical disk can be broken up into multiple *partitions, slices, or mini-disks*, each of which becomes a virtual disk and can have its own filesystem
 - Multiple physical disks can be combined into one *volume*, i.e. a larger virtual disk, with its own filesystem spanning the physical disks.



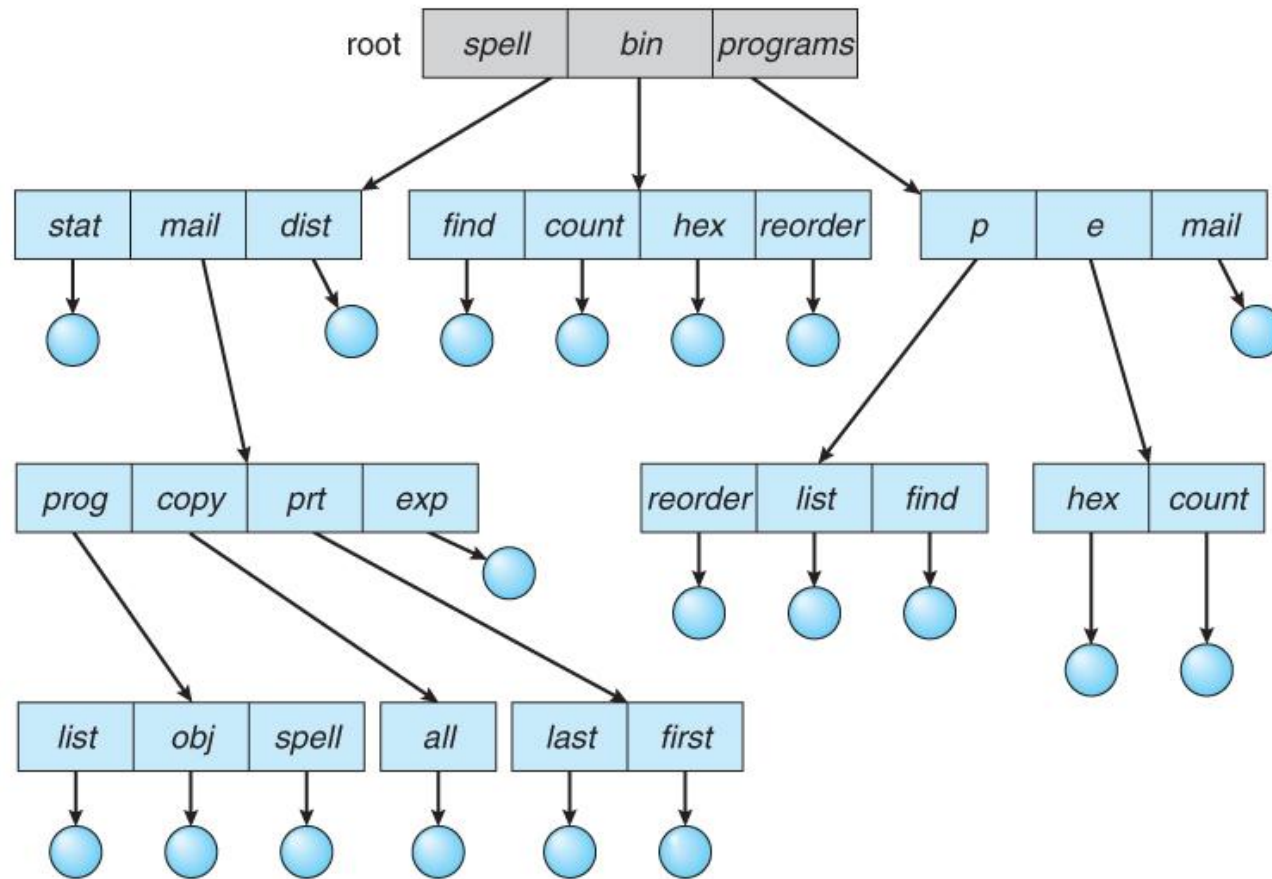
Directory Overview

- Directory operations - Search, create, delete, list, rename, traverse
- Single-Level Directory - easy to implement, but each file must have a unique name
- Two-Level Directory - each user-> own directory; Unique file name within the user's; Master file directory to keep track; separate directory->system
 - If access to other directories is allowed, then provision must be made to specify the directory being accessed.
 - If access is denied, then special consideration must be made for users to run programs located in system directories.
 - A *search path* is the list of directories in which to search for executable programs, and can be set uniquely for each user.



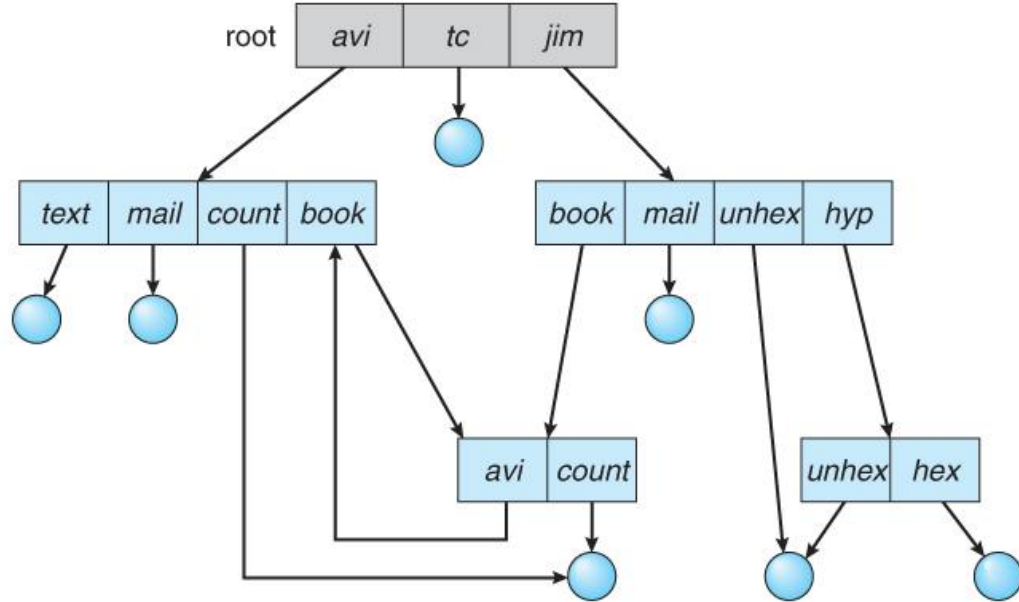
Tree-Structured

- Extension to the two-tiered directory
- Each user / process has the concept of a *current directory* from which all (relative) searches take place.
- Files may be accessed using either absolute pathnames (relative to the root of the tree) or relative pathnames (relative to the current directory.)
- Directories are stored the same as any other file in the system, except there is a bit that identifies them as directories, and they have some special structure that the OS understands.
- Whether or not to allow the removal of directories that are not empty - Windows requires that directories be emptied first, and UNIX provides an option for deleting entire sub-trees.



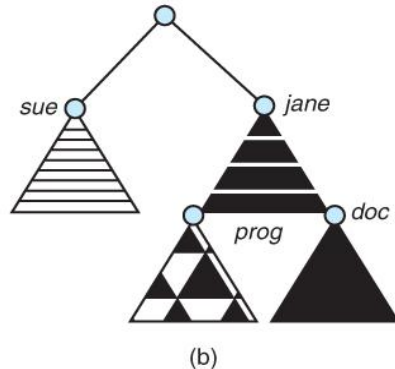
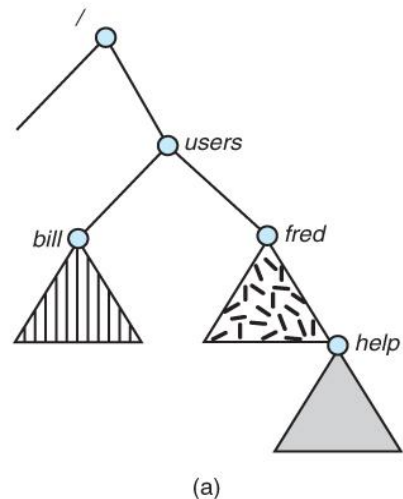
General Graph Directory

- If cycles are allowed in the graphs, then several problems can arise
- Search algorithms can go into infinite loops.
 - Do not follow link (or symbolic link) in search
- Sub-trees can become disconnected from rest of the tree and still not have their reference counts reduced to zero.
 - Periodic garbage collection is required to detect

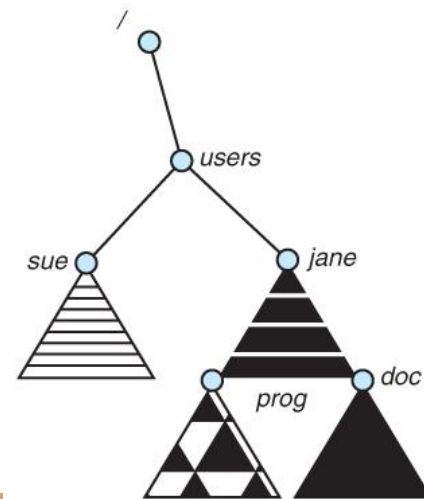


File System Mounting

- Combine multiple file systems into one large tree structure.
 - The mount command is given a filesystem to mount and a *mount point* (directory) on which to attach it.
 - Once a file system is mounted onto a mount point, any further references to that directory actually refer to the root of the mounted file system.
 - Any files (or sub-directories) that had been stored in the mount point directory prior to mounting the new filesystem are now hidden by the mounted filesystem -> Mount on empty directories
- Filesystems can only be mounted by root -> Can be Read-only, other restrictions
 - Anyone can run the mount command to see what filesystems are currently mounted.



- Traditional Windows OS -> extended two-tier directory structure,
- First tier separates volumes by drive letters, and a tree structure is implemented below that level.
- Mac similar, Unix -> Mount to any dir



File Sharing

- Multiple Users -> more information stored for each file ->
 - The owner-> own and control; The group of other user IDs that may have some special access to the file;
 - What access rights are afforded to the owner (User), the Group, and to the rest of the world (the universe, a.k.a. Others.) ; Various complicated access control.
- Remote File Systems -> Remote Computer -> Originally ftp, access control or anonymous -> WWW access files on remote systems without mounting
 - *distributed file systems* allow remote file systems to be mounted onto a local directory structure, and accessed using normal file access commands.
 - Client-Server Model - mounts a filesystem that is physically located on another system-> a *server*, and the system which mounts them is the *client*.
 - *Security Concerns and Network File System*
 - Distributed Information Systems - **Domain Name System, DNS**, provides for a unique naming system across all of the Internet.
 - Domain names are maintained by the *Network Information System, NIS*
 - *New Approach - Lightweight Directory-Access Protocol, LDAP*,
 - Failure Modes - When a local disk file is unavailable ->generally non-recoverable.
 - a remote file is unavailable, there are many possible reasons, and whether or not it is unrecoverable is not readily apparent. Hence most remote access systems allow for blocking or delayed response, in the hopes that the remote system (or the network) will come back up eventually.

Consistency Semantics

- *Consistency Semantics* deals with the consistency between the views of shared files on a networked system. When one user changes the file, when do other users see the changes?
 - long delays involved in network operations prohibit the use of atomic operations
- **UNIX Semantics** - Writes to an open file are immediately visible to any other user who has the file open.
 - Another uses a shared location pointer, which is adjusted for all sharing users.
- **Session Semantics** - Andrew File System - Write is not immediately visible.
 - When it is closed, changes become available to other users
 - Access control is maintained through (somewhat) complicated access control lists, which may grant access to the entire world (literally) or to specifically named users accessing the files from specifically named remote environments.
- **Immutable Shared File Semantics** - When a file is declared as *shared* by its creator, it becomes immutable and the name cannot be re-used for any other resource. Hence it becomes read-only, and shared access is simple.

File Protection

- Simple - remove all access to the file -> Unusable
- Type of Access - Read - view the content, Write - change the content, Execute - load to CPU and follow instructions, Append - add to the end, Delete - remove, List - view the name and other attribute
 - Higher level operation - Ex . copy
- Access Control - complicated *Access Control Lists, ACL*, - > exactly what access is allowed or denied for specific users or groups - AFS, Distributed Access
 - Windows - GUI
- UNIX - 9 access control bits, in three groups of three. These correspond to R, W, and X permissions for each of the Owner, Group, and Others
- Special Bits - The sticky bit on a directory modifies write permission, allowing users to only delete files for which they are the owner.
 - The numeric form of chmod is needed to set these advanced bits.
 - The set user ID (SUID) bit and/or the set group ID (SGID) bits applied to executable files temporarily change the identity of whoever runs the program to match that of the owner / group of the executable program.
- Passwords. Older systems must be *retrofitted* to share files on a network.